

Dragonn

Make the Hexagon NPU reachable for models nobody converted for you — and prove, on the laptop itself, that the NPU is really what ran them.

Snapdragon® AI Lab Build & Present Challenge 2026 · Rishi (RishiMaara) · github.com/RishiMaara/Dragonn · MIT licensed

The problem

A Snapdragon X Elite laptop ships with a 45 TOPS Hexagon NPU. For any model that isn't already in a vendor catalog — a fine-tune, a newer checkpoint — getting it onto that NPU fails in ways that produce **no error message**:

- The standard quantization recipe (`quantize_dynamic`) cost MobileNetV2 **72 points of accuracy** on a real X Elite, and crashed the device runtime outright on Whisper-base.
- ONNX Runtime's documented provider argument silently yields a **CPU-only session** while the application logs say NPU.
- A model can pass every local check and still be **rejected by the chip** — in this project's case over one sign bit, on one input, of one operator type.

Every one of those failures is silent. The application works. The battery doesn't.

What I built

Component	What it does
Scanner	Answers "will this model run on the Hexagon NPU, and if not, which operators and why" on any Windows PC, with no Snapdragon device present: static operator and precision rules first, then Qualcomm's own HTP compiler run locally in about four seconds.
Converter	Static a16w8 QDQ quantization done the QNN-specific way, calibrated on real data, including the fixes that appear in no quickstart guide.
Validator	Runs the model on a real Snapdragon X Elite through Qualcomm AI Hub and reconciles the prediction against what the device actually did.
Application	Offline speech-to-text server (OpenAI-compatible API) and dashboard that report the execution provider that actually attached — never the one requested.

It composes what already exists rather than reinventing it: ONNX Runtime's strict no-CPU-fallback session, Microsoft Olive's attention-mask surgery, Qualcomm's own mask convention and AI Hub models. The gap it fills is **diagnosis** — ONNX Runtime's strict mode answers yes or no, AI Hub needs a cloud upload per model, and Olive applies a fix only once you already know which one you need.

Verified on real Snapdragon hardware

Device: Snapdragon X Elite CRD (SC8380XP, Hexagon v73) via Qualcomm AI Hub. Every number below is committed as machine-readable evidence in the repository, indexed claim by claim, together with the device's own logs.

Claim	Measured	Evidence
Encoder runs on the NPU	185 / 187 layers	aihub_validation.json
Latency, 100 runs	17.8 ms median	profile_jpxlee83p

A quantized model that misses the NPU is slower than not quantizing at all	0.79× / 0.72×	two separate sessions
Quantization costs no accuracy on CPU (57 held-out clips)	12.70% WER = FP32	wer_calibration_and_precision.json
The NPU costs a little accuracy	14.07% WER	sign test p ≈ 0.29
Skipping quantization is not a shortcut: FP32 passes locally, fails on the device	error 6000, twice	aihub_fp32_on_htp_FAILED.json
Rerunning the pipeline rebuilds the exact validated model	bit-for-bit	SHA-256 pinned

It isn't one model

Five more popular models through both the naive path and this pipeline, each then profiled on the same X Elite — and all five land on the NPU.

Model	Use on a laptop	FP32 → naive → this pipeline	On the NPU
MobileNetV2	Image classification	79.5% → 7.2% → 79.5% top-1	0.36 ms
Whisper-base	Speech recognition	8.84% → 11.33% → 8.84% WER	45.7 ms
MiniLM-L6	Embeddings, local search	100% → 63.3% → 97.3% same top hit	1.30 ms
DistilBERT SST-2	Sentiment	90.7% → 90.7% → 91.0%	2.44 ms
CLIP ViT-B/32	Image search	98.2% → 98.8% → 98.2% zero-shot	2.55 ms

MobileNetV2 is the clearest case of the trap this project exists for: quantized the naive way it passes every *placement* check — one NPU graph, nothing left on CPU — and classifies 7% of images correctly. Weight granularity decided two of these five: per-tensor weights also cost MiniLM a quarter of its top search hits while holding an embedding cosine of 0.987. Per-channel weights fixed both, at a measured cost of 0.01 ms on the device — and nothing in the placement toolchain tells you any of it.

Against Qualcomm's own NPU Whisper

Qualcomm publishes Whisper-Tiny pre-compiled for each chipset. Running their X Elite build on the same device, through the same harness: their encoder **25.0 ms**, this pipeline's **17.8 ms**. Their decoder — the piece this project does not yet run on the NPU — is **2.60 ms per token**, 509 of 509 layers on the NPU. Read honestly: the two models are not the same graph (Qualcomm restructures attention and stays in float, this pipeline quantizes), so this shows a general-purpose conversion path landing in the same league as the vendor's hand-optimised build, not a benchmark win.

Running it on a Snapdragon PC

One command: `powershell -ExecutionPolicy Bypass -File .\setup-snapdragon.ps1`. It refuses to pass quietly — it checks for ARM64-native Python (x64 Python under emulation can never load `QnnHtp.dll`), installs the device dependencies, confirms ONNX Runtime can see the Hexagon NPU and names the driver version if it can't, then builds a session with CPU fallback *disabled*, so "runs on the NPU" is proven on that laptop rather than assumed. The pre-flight scanner itself needs no Snapdragon hardware at all.

Scope, and how to check it

What runs where today: the encoder runs on the Hexagon NPU and the decoder on CPU, so every latency in this document is an encoder latency. The decode loop for an NPU decoder is implemented and verified against all nineteen inputs the shipped decoder model declares, and its device numbers are measured (2.60 ms per token, 509 of 509 layers on the NPU); running it end to end requires a physical Snapdragon, because that model is an EPContext graph only QNN EP can load.

How to check any number here: `models/reports/README.md` indexes every claim against the file that backs it — machine-readable results, the profiling job that produced each one, and the device's own runtime logs, including the runs that failed and what the chip said when it refused a model. `python -m tools.model_zoo` rebuilds the five-model comparison from scratch and `python -m scanner --input <model> --compile-check` checks any model you bring, on any Windows PC.

On the speed comparison: the 24–57× figure is measured against the cloud service's own CPU baseline, whose thread configuration it does not disclose. Treat it as the ratio against that baseline rather than against a hand-tuned CPU.

Repository: github.com/RishiMaara/Dragon · MIT licensed · 49 automated tests (48 passing, 1 skipped for lack of a physical device), each pinning a bug that once produced a wrong number, run on every push (Ubuntu, and Windows with the QNN plugin). Every claim in this document traces to a committed file under `models/reports/`.